



Diretrizes de Infraestrutura para o Trino

Parâmetros de referência para a configuração de um cluster Trino em Kubernetes: dimensionamento de memória e JVM, execução tolerante a falhas, spill, concorrência, segurança base e preparação da integração com o Apache Ranger.

Universidade de Brasília

Lab Livre · Faculdade do Gama, UnB

Ministério da Cultura

Gov Hub BR

Projeto de Pesquisa

Plataforma de Dados do Ministério da Cultura · Gov Hub BR

Meta 03 · Produto 1

Motor de governança e autorização de acesso a dados.

Documento

Diretrizes de Infraestrutura para o Trino



Índice

01	Objetivo e escopo	4
	Pontos considerados	4
02	Coordinator, workers e o heap	5
	Separação entre coordinator e workers	5
	O modelo de memória	5
	Os quatro parâmetros principais	7
	Referência de dimensionamento da JVM	7
03	Execução tolerante a falhas e troca	8
	Bucket ou volume compartilhado	8
04	Descarregar memória em disco	10
05	Concorrência e buffers	12
	Paralelismo interno	12
	Buffers da área de troca	12
06	Segurança base antes do Ranger	13
	Grupos de recurso	13
07	Recomendações finais	15
	Ambientes pequenos e de homologação	15
	Produção	15





Este documento apresenta diretrizes técnicas para a configuração de um cluster Trino em Kubernetes, com foco em dimensionamento de memória, JVM, gerenciador de troca, spill, concorrência, segurança base e preparação da futura integração com o Apache Ranger.

PARÂMETROS DE REFERÊNCIA, NÃO CONFIGURAÇÃO FINAL

O documento **não define uma configuração final de ambiente**. Os valores apresentados devem ser tratados como parâmetros de referência, a serem ajustados conforme o volume de dados, a quantidade de workers, o perfil das consultas e a capacidade real da infraestrutura.

Pontos considerados

- Separação entre coordinator e workers.
- Configuração de memória de consulta.
- Dimensionamento da JVM.
- Uso do gerenciador de troca (*exchange manager*).
- Uso de bucket ou volume compartilhado para a troca.
- Uso de spill.
- Configuração de concorrência.
- Segurança base anterior à integração com o Ranger.
- Uso futuro de grupos de recurso.

As configurações do Apache Ranger propriamente ditas não integram este escopo: o documento prepara o terreno para elas.

Coordinator, workers e o heap



Separação entre coordinator e workers

Recomenda-se manter o coordinator fora da execução de tarefas pesadas de consulta.

```
node-scheduler.include-coordinator=false
```

Com essa configuração, o coordinator permanece dedicado ao planejamento, à coordenação e ao agendamento das consultas, e o processamento distribuído concentra-se nos workers. A separação reduz o risco de degradação do coordinator em cenários de alta concorrência ou de consultas sobre grande volume de dados.

O modelo de memória

O Trino executa sobre JVM. O limite de memória do contêiner não deve, portanto, ser confundido com o heap máximo da JVM. O parâmetro `Xmx` define apenas o tamanho máximo do heap; o processo Java também consome memória fora dele — buffers diretos, metaspace, cache de código, pilhas de threads, bibliotecas nativas e buffers de entrada e saída.

Modelo de memória de um worker

O limite de memória do contêiner não se confunde com o heap máximo da JVM

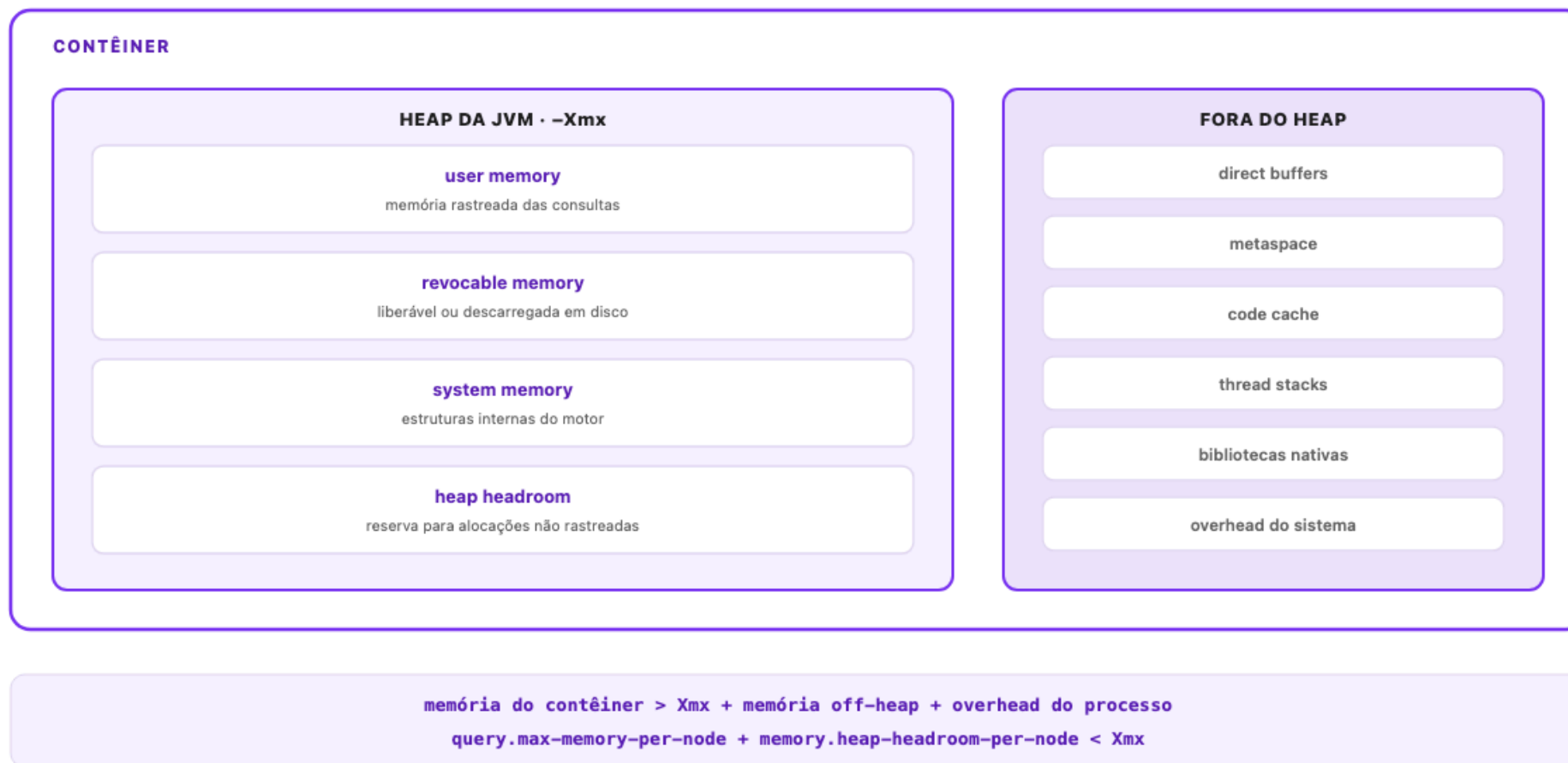


Figura 1. Modelo de memória de um worker. Dentro do heap da JVM convivem a memória de usuário, a revogável, a de sistema e a reserva de headroom; fora dele ficam os componentes que o gerenciador de memória do Trino não rastreia. As duas relações ao pé governam todo o dimensionamento.

AS DUAS RELAÇÕES QUE GOVERNAM O DIMENSIONAMENTO

A memória do contêiner precisa ser maior que a soma do `Xmx`, da memória fora do heap e do overhead do processo. E, dentro do heap, a soma de `query.max-memory-per-node` com `memory.heap-headroom-per-node` precisa ser menor que o `Xmx`. A documentação do Trino define o headroom como reserva, dentro do heap, para alocações que o gerenciador de memória não rastreia.

Os quatro parâmetros principais

Parâmetro	O que define	Valor de referência
<code>query.max-memory-per-node</code>	Limite de memória de usuário que uma única consulta pode consumir em cada worker. Deve acompanhar o heap disponível; em workers pequenos, precisa ser reduzido para evitar pressão excessiva.	4GB
<code>query.max-memory</code>	Limite de memória de usuário da consulta no cluster inteiro. Deve acompanhar a quantidade de workers e o limite por nó — num cluster pequeno não faz sentido um teto global muito acima da soma prática da capacidade.	24GB
<code>query.max-total-memory</code>	Limite total da consulta no cluster, incluindo a memória revogável. Precisa ser maior que o anterior, porque parte da execução pode usar memória revogável, liberável ou descarregável em disco.	48GB
<code>memory.heap-headroom-per-node</code>	Margem de segurança dentro do heap, e não memória fora dele, reservada às alocações não rastreadas diretamente.	2GB

Referência de dimensionamento da JVM

Memória do worker	Xmx sugerido	Headroom	Memória por consulta e nó
8 GiB	5G a 6G	1GB	1GB a 2GB
16 GiB	11G a 12G	2GB	3GB a 4GB
32 GiB	24G a 26G	4GB	6GB a 8GB
64 GiB	48G a 52G	8GB	12GB a 16GB

A tabela é referência inicial, e não regra fixa. O ajuste final depende da concorrência, do tipo de consulta, dos conectores utilizados, do volume de dados e do comportamento efetivamente observado em produção.





Para cargas em lote ou consultas de maior duração, recomenda-se avaliar a execução tolerante a falhas. Com a política de repetição por tarefa, o Trino pode reexecutar tarefas individuais de uma consulta em caso de falha, em vez de reiniciar a consulta inteira — razão pela qual a documentação a recomenda para consultas em lote de grande porte.

```
retry-policy=TASK
```

Nesse modo, é obrigatório configurar um **gerenciador de troca**, responsável por armazenar os dados intermediários da execução. Esses dados precisam estar acessíveis a todos os workers envolvidos na consulta.

```
server:
  exchangeManager:
    name: "filesystem"
    baseDir:
      - "s3://trino-exchange/spooling"
```

Bucket ou volume compartilhado

Para produção, a alternativa mais adequada costuma ser o armazenamento de objetos. Ele desacopla a área de troca do ciclo de vida dos pods, facilita o acesso compartilhado por todos os workers, permite distribuir carga em múltiplos buckets ou prefixos, reduz a dependência de um volume centralizado e tende a ser mais adequado à execução tolerante a falhas.

Um volume de leitura e escrita múltiplas pode ser usado, desde que todos os workers alcancem o mesmo diretório. A limitação principal não é de compatibilidade, mas de capacidade de entrada e saída: a troca pode gerar grande volume de leitura e escrita, e um backend compartilhado limitado torna-se gargalo.

Ambiente	Recomendação
Desenvolvimento e homologação	Volume compartilhado pode ser suficiente
Produção pequena	Volume compartilhado é possível, desde que medido
Produção com carga em lote relevante	Preferir armazenamento de objetos
Em qualquer caso	Evitar misturar a área de troca e a de spill no mesmo volume





O spill é o mecanismo de descarregar dados intermediários da memória para o disco. Permite executar consultas de maior consumo de memória, ao custo de degradação de desempenho. O Trino documenta suporte a spill em agregações, junções, ordenações e funções de janela.

```
spill-enabled=true  
spiller-spill-path=/data/spill  
max-spill-per-node=100GB  
query-max-spill-per-node=50GB  
spill-compression-codec=ZSTD  
spill-encryption-enabled=true
```

Parâmetro	O que define	Consideração
<code>spill-enabled</code>	Habilita o descarregamento em disco	Recomendado quando há risco de estouro de memória em junções, ordenações, agregações e funções de janela
<code>spiller-spill-path</code>	Caminho de gravação dos arquivos	Deve estar em volume dedicado e preferencialmente rápido. Não convém usar o disco do sistema nem o mesmo dos registros da JVM: a saturação do disco causa pausas e falhas de consulta
<code>max-spill-per-node</code>	Limite total por worker, somadas todas as consultas	Precisa ser compatível com a capacidade real do volume
<code>query-max-spill-per-node</code>	Limite de uma única consulta num worker	Evita que uma consulta consuma todo o espaço de spill do nó
<code>spill-compression-codec</code>	Compressão dos dados gravados	Reduz uso de disco e de entrada e saída; aumenta o consumo de processador
<code>spill-encryption-enabled</code>	Criptografia dos arquivos	Recomendado quando as consultas processam dados sensíveis, pessoais ou regulados; aumenta o consumo de processador e pode reduzir a vazão

POR QUE A CRIPTOGRAFIA DO SPILL IMPORTA AQUI

A plataforma processa documentos de identificação e autodeclaração de raça e de deficiência de agentes culturais. Uma consulta que envolva esses atributos e recorra a spill grava dados sensíveis em disco. A criptografia do spill é, nesse contexto, medida de proteção do dado em repouso, e não apenas ajuste de desempenho.





Paralelismo interno

```
task.concurrency=4
```

O parâmetro controla o paralelismo interno da execução das tarefas. Valores maiores podem aumentar a vazão em consultas grandes, mas também elevam o consumo de processador e de memória; valores menores tendem a ser mais seguros em ambientes de alta concorrência ou com workers pequenos.

- Workers pequenos: iniciar em 4.
- Workers maiores: testar 4, 8 ou 16 com aferição.
- Não aumentar sem medir processador, memória, tempo de consulta e impacto sobre a concorrência.

Buffers da área de troca

```
exchange.sink-buffer-pool-min-size=16  
exchange.sink-buffers-per-partition=4  
exchange.source-concurrent-readers=4  
sink.max-buffer-size=128MB
```

Esses parâmetros afetam o uso de buffers e a leitura concorrente na área de troca. Convém manter valores conservadores no início e aumentá-los apenas diante de gargalo comprovado, acompanhando o uso de memória, a entrada e saída e o tempo de leitura e escrita no armazenamento.

Segurança base antes do Ranger



A integração com o Apache Ranger pressupõe uma camada de segurança já estabelecida no cluster. Preparar essa camada antes reduz retrabalho na configuração das políticas.

O primeiro item é o segredo compartilhado que protege a comunicação interna entre coordinator e workers em cenários com autenticação ou TLS interno.

```
internal-communication.shared-secret=<segredo-longo>
```

Recomenda-se ainda definir previamente um conjunto de convenções que as políticas de autorização tomarão como referência. São decisões baratas de tomar antes e caras de mudar depois, porque cada uma delas aparece no texto das políticas:

- Nomes estáveis para os catálogos.
- Padrão de autenticação.
- Mapeamento de usuários e de grupos.
- Separação entre usuários humanos, serviços e rotinas automatizadas.
- Convenção de schemas e de permissões.

Grupos de recurso

Os grupos de recurso devem ser considerados antes da entrada em produção multiuso, para evitar que cargas de visualização, de processamento em lote e de administração concorram sem controle pelos mesmos recursos.

Grupo	Perfil
Consulta interativa	Menor latência, menor fila e limite de memória mais restrito
Processamento em lote	Consultas longas, maior tolerância a fila, menor concorrência e limite de memória mais alto
Administração	Acesso restrito, limites específicos e uso operacional



A separação reduz o risco de uma carga em lote degradar consultas interativas, ou de múltiplas consultas de visualização impedirem execuções de engenharia de dados.





Ambientes pequenos e de homologação

- Manter o coordinator fora do processamento.
- Usar `Xmx` abaixo do limite de memória do contêiner.
- Reservar o headroom do heap.
- Usar volume compartilhado para a área de troca apenas se o backend suportar a carga.
- Usar spill em volume separado.
- Iniciar com paralelismo conservador.
- Medir processador, memória, entrada e saída e tempo de consulta antes de escalar parâmetros.

Produção

- Preferir armazenamento de objetos para a área de troca.
- Evitar a área de troca em disco efêmero.
- Evitar troca e spill no mesmo volume.
- Usar spill em volume dedicado por worker.
- Configurar grupos de recurso.
- Configurar o segredo compartilhado e a autenticação antes do Ranger.
- Manter nomes de catálogos estáveis.
- Monitorar JVM, coleta de lixo, memória de consulta, spill e entrada e saída da área de troca.

O QUE ESTE DOCUMENTO HABILITA

As diretrizes aqui reunidas constituem o preparo técnico de dois compromissos do relatório a que este anexo acompanha: a implantação do componente de governança e autorização de acesso, previsto no Produto 1 da Meta 03, e a oficina de capacitação sobre governança descentralizada de dados, prevista no Produto 2 da Meta 05, cujo escopo recai justamente sobre o motor de consulta distribuído e sobre esse componente.

